1. Architectural decision

13 November 2026. Exact-real formalization; no training, GPU run, or speedup claim.

The conditional decide-then-propagate pattern survives. Once the complete read/write schedule is fixed, attention is linear in values, and index reads plus the actual phase-one slot-store writes compose in the same affine-operator monoid on an explicit source-state/output product. A closed block triangular summary suffices. This is a replay theorem; it does not parallelize the computation of the schedule.

Lean witnesses: `A.readout_linear; A.step_eq_embed; Ev.combined_closed_summary; Ev.combined_routed_scan; E.store_index_single_scan`

Two stronger design claims fail. Candidate-normalized truncation can sharpen a surviving atom and change output scale. Recomputing strengths, scales, support, or nonlinear values from the scanned state can destroy affinity. Adding read outputs also leaves the diagonal family on the same coordinates, although a larger closed triangular family exists.

Lean witnesses: `O.truncation_sharpens_survivor; O.truncation_changes_readout_scale; O.state_dependent_weights_not_affine; O.state_dependent_alpha_not_affine; O.state_dependent_support_not_affine; O.nonlinear_value_source_not_affine; O.read_not_diagonal; RW.toAffine_thenOp`

Obligation	Formal verdict
Measure operator	Finite nonnegative atomic measures; exact full-population mass and support metadata; generic universes and explicit value sources.
Central affinity	Proved for fixed coefficients and support, with a linear or supplied constant value source.
Additive laws	Budget bound, zero ablation, and empty behavior proved. Truncation-invariant survivor weights require the full-population denominator.
Combined scan	Sequential, chunk, prefix, and ordered-tree execution agree under one supplied schedule.
Boundary	Fixed joint normalization and fixed learned α remain linear. State dependence, fixed support caps, and omitted coordinates have named obstructions.

Citation convention. All short names below expand under `MemoryAlgebra.Attention`: **N** = `Neighborhood`; **A** = `AttentionOperator`; **M** = the namespace itself; **RW** = `ReadWriteSummary`; **Ev** = `Event`; **E** = `Emender`; **O** = `Obstructions`. Definitions are explicitly identified. Mathematical assertions are accompanied by Lean witnesses; architectural and implementation requirements are labeled assumptions or limitations. The source design is `experiments/expressivity_tasks/x43_fork_resolved/DESIGN_SUFFIX_ATTENTION.md`, Revision 2.

2. Complete populations, returned candidates, and measures

Definitions (N, A). For an arbitrary position universe U and owner universe R, a `Neighborhood U R` consists of a finite eligible set E, weights $w : U \to \mathbb{R}$ with $w(j) \geq 0$, and an owner map $U \to R$. Positions may be context locations, corpus locations, or store slots. Finiteness is per query; U need not be finite. An `AttentionOperator U R V` additionally fixes a returned subset $C \subseteq E$, a nonnegative scale α, and a normalization policy. The type V is a real module of values, explicitly part of the operator type.

$$M_E = \sum_{j \in E} w(j), \quad M_C = \sum_{j \in C} w(j).$$

Definitions (N.measure, A.measure). The raw and returned measures are genuine `MeasureTheory.Measure` values on any supplied measurable space:

$$\nu = \sum_{j \in E} w(j)\delta_j, \quad \mu = \sum_{j \in C} a(j)\delta_j.$$

The Lean measure coefficients are `ENNReal.ofReal` of these nonnegative reals. Their total masses are exactly the corresponding finite sums and are finite. The readout is defined as the finite atomic weighted sum in V; no general Bochner-integral identification theorem is claimed.

Lean witnesses: `N.mass_nonneg; N.measure_univ; N.measure_finite; A.coefficient_nonneg; A.measure_univ; A.measure_finite`

Definition (A.exactAggregates). The answer carries four full-population quantities: raw total mass, eligible position count, positive-weight position count, and distinct eligible owner count. Zero-weight eligible positions count in eligibility and owner support but not positive support. These definitions deliberately distinguish candidate count, positive support, and distinct provenance support.

Two schedules with the same population have identical exact aggregates, irrespective of their returned candidates, α, or normalization policy. Empty eligibility has zero raw mass, zero eligible count, and no owners. Counts describe the supplied population; they are not estimates obtained by extrapolating a capped list.

Lean witnesses: `A.aggregates_independent_of_truncation; N.empty_aggregates`

Definitions (E.Position, E.SourceState, E.values). A tagged union combines SELF, WORLD, and STORE positions. The source state contains the phase-one numeric store and explicit context/world value tables. The three source equations select exactly the appropriate table or slot value. The same `AttentionOperator` and `MemoryOperator` interface applies to all three; heterogeneous mixtures also permit separate universes.

Lean witnesses: `E.attend_self_source; E.attend_world_source; E.attend_store_source; M.heterogeneous_mixture_linear`

> **Assumption / unproved refinement.** Eligibility, source-document exclusion, causal visibility, distinct-seed scoring, generation pinning, and correct owners must be established by the producer. MEM search and chaining are not implemented or verified here. Exact aggregate definitions do not establish that an index can compute them without enumeration. Entropy, branch histograms, and provenance deduplication are outside this formal record.

3. The central theorem: linear in values under a schedule

Definitions (A.denominator, A.coefficient, A.readout). Two policies are modeled:

$$Z = \begin{cases} M_C \text{ retained} \\ M_E \text{ full} \end{cases}, \quad a(j) = \alpha \frac{w(j)}{Z}, \quad R(v) = \sum_{j \in C} a(j) v(j).$$

Lean's total real division assigns zero to division by zero. The formal semantics therefore have an explicit zero-mass outcome, proved below. An implementation needs an equivalent zero-denominator branch; floating-point `0/0` is not this semantics.

Theorem. With the schedule fixed, R is a real-linear map from value tables to values, so for all tables x, y and all real c,

$$R(cx + y) = cR(x) + R(y), \quad R(cx + (1-c)y) = cR(x) + (1-c)R(y).$$

Proof sketch. Distribute scalar multiplication and addition into each summand, then distribute them through the finite sum. Neither positivity nor normalization is needed for this linearity argument; they are needed for the mass-budget laws. The proof is packaged as `A.readoutLinear`, not merely an equality on one trajectory.

Lean witnesses: `A.readout_linear; A.readout_affine`

Precisely fixed. The eligible population and owner identities, returned support, all numerical weights, α, the normalization choice and thus its denominator, and all source-selection/projection maps are supplied schedule data. Metadata is pinned for reproducibility; the linearity argument only uses fixed support, coefficients, and source maps. α may have been learned; that history does not change the fixed-coefficient theorem. Values may vary arbitrarily in the theorem. For a read affine in the carried state, those values must be a fixed affine function of that state or supplied constants.

Lean witnesses: `O.fixed_learned_alpha_linear; Ev.step_eq_summary; O.nonlinear_value_source_not_affine`

Definition (A.step). Lift the read to an endomorphism of the table/residual state:

$$(v, r) \mapsto (v, r + R(v)).$$

Its linear part is this map and its affine injection is zero. The direct step equals the affine embedding on every state. This supplies the existing `MemoryOperator` class, and any fixed list is scan compatible relative to that schedule.

Lean witnesses: `A.step_eq_embed; A.fixed_schedule_scan`

A requested boundary claim is false. Joint normalization across operators is also linear in values when all its weights and support are fixed. The generic map `O.fixedJointLinear` divides fixed weights by their fixed total. A fixed learned α likewise does not break affinity. Joint normalization can still starve a branch: weights 1 and 99 give the first branch mass 1/100 instead of its independent budget 1.

Lean witnesses: `O.fixed_joint_normalization_linear; O.fixed_learned_alpha_linear; O.joint_normalization_starvation`

4. Additive budgets, empty support, and exact ablation

Mass laws. For both normalization policies, $M_C \geq 0$, $M_C \leq Z$, and

$$m = \sum_{j \in C} a(j) = \alpha \frac{M_C}{Z} \leq \alpha.$$

For retained normalization with $M_C \neq 0$, $m = \alpha$. If the denominator is zero, every coefficient and output mass are zero. With bounded values $\|v(j)\| \leq B$ and $B \geq 0$, the norm of a branch's contribution is at most αB.

Lean witnesses: `A.retainedMass_nonneg; A.retainedMass_le_denominator; A.outputMass_formula; A.outputMass_le_alpha; A.retained_normalization_mass; A.zero_denominator; M.readout_norm_le`

Proof sketch of the norm bound. Use the triangle inequality on the finite sum, replace each coefficient's absolute value with itself by nonnegativity, bound each value norm by B, then use the mass budget. This upper bound is independent of the candidate cap. It is not equality of outputs, and it needs bounds on values and α to become a uniform architecture-wide bound. There is a counterexample to a norm bound with unrestricted values even at α=1 and with one returned atom.

Lean witnesses: `M.readout_norm_le; O.no_bound_without_bounded_values`

Additive mixtures. Measures and readouts add. Mixture total mass is at most the sum of the scales; if every candidate-normalized branch has positive retained mass, its individual mass remains its own α. With the same value bound, the mixture norm is at most $B \sum_o \alpha_o$. Read-only additions on an unchanged source commute; their affine product adds their readouts. Interleaved writes retain chronological order in the scan theorem.

Lean witnesses: `M.mixture_measure_mass; M.mixture_mass_budget; M.independent_positive_budgets; M.mixture_linear; M.heterogeneous_mixture_linear; M.mixture_disjoint_union; M.mixture_norm_budget; M.attention_addition_commutes; M.attention_product_readout`

Zero is exact. α=0 makes the returned measure zero, the readout zero, and the embedded memory operator the identity. Removing that branch from an additive mixture leaves the result unchanged. Empty returned support contributes zero mass and zero readout. Empty eligible support forces empty returned support; nonempty support whose total raw weight is zero also gives zero readout.

Lean witnesses: `A.alpha_zero_measure; A.alpha_zero_readout; A.alpha_zero_noop; A.alpha_zero_identity; M.ablation_exact; A.empty_candidates; A.empty_support; A.zero_mass_readout`

The converse needs hypotheses. A positive α on empty support is also a no-op. For scalar values and positive retained mass, α=0 is equivalent to every possible readout being zero. A zero readout on one particular table can instead reflect cancellation or zero values; it does not identify α.

Lean witnesses: `O.positive_alpha_can_be_noop; A.alpha_zero_iff_all_readouts_zero; A.readout_one_eq_outputMass; O.cancellation_with_positive_alpha`

5. Truncation: the false claim and the available repair

Take two eligible atoms, each with raw weight 1, distinct owners, α=1, and values 0 and 2. Compare returning both atoms with returning only the atom valued 2.

Policy / candidates	Survivor weight	Output mass	Readout
Retained / both	1/2	1	1
Retained / one	1	1	2
Full / both	1/2	1	1
Full / one	1/2	1/2	1

Lean witnesses: `O.truncation_changes_readout_scale; O.full_normalization_readout; O.full_normalization_loses_omitted_mass; A.retained_normalization_mass`

The retained-denominator formula in R2.4 doubles the survivor's coefficient and the readout norm. Returning an exact aggregate alongside it does not prevent this rescaling. Thus "truncation only drops candidates" and "never makes an operator sharper" are false for that formula. The total nonempty branch budget is preserved; the distribution over candidates and the readout scale need not be.

Lean witnesses: `O.truncation_sharpens_survivor; O.truncation_changes_readout_scale; A.retained_normalization_mass`

The full-population denominator gives a different, precisely characterized law. For any two truncations of the same population and scale, every survivor retains exactly the same coefficient. Restricting the candidates can only decrease total output mass. The larger readout equals the smaller readout plus the weighted sum over omitted atoms, with no redistribution to survivors.

Lean witnesses: `A.full_coefficients_truncation_invariant; A.full_truncation_mass_monotone; A.full_truncation_decomposition; O.full_normalization_loses_omitted_mass`

Proof sketch. Under the full policy, the expression $\alpha \frac{w(j)}{M_E}$ contains no returned-candidate set. Split the larger finite sum into retained and omitted sets; monotonicity of its scalar mass follows because omitted coefficients are nonnegative. The value sum itself need not be monotone because values are vectors.

There is no way, in the two-atom example, to drop the other atom, keep the survivor's coefficient 1/2, and preserve total mass 1. These goals require different normalization contracts. Both policies keep the exact population metadata unchanged.

Lean witnesses: `O.no_preserved_mass_and_survivor; A.aggregates_independent_of_truncation`

Architectural recommendation, not a proved requirement. Choose the intended contract explicitly. Retained normalization keeps a nonempty branch at its α budget and discloses truncation, while allowing sharpening. Full normalization implements literal dropping without sharpening and discloses lost mass. Neither policy proves that the retained candidates are representative or that predictions are unchanged. This formalization models both and does not silently rewrite R2.4.

6. Combined reads and writes: one scheduled scan

Definitions (Ev, RW). Let S be the source state and V an output/residual space. An event either applies an existing `MemoryOperator` write to S, or reads a fixed attention schedule with values $P(s) + q$, where $P : S \to (U \to V)$ is linear and q is fixed. The direct read step is $(s, r) \mapsto (s, r + R(P(s) + q))$.

A summary consists of a store affine map (L, b), a read linear map $H : S \to V$, and a bias $c \in V$, acting as

$$(s, r) \mapsto (Ls + b, r + Hs + c).$$

Two summaries in chronological order compose as

$$L = L_2 \circ L_1, \quad b = L_2 b_1 + b_2,$$

$$H = H_1 + H_2 \circ L_1, \quad c = c_1 + H_2 b_1 + c_2.$$

Proof sketch. Substitute the first output into the second. Linearity distributes the second read over the first store carry and injection. The resulting summary's affine embedding is exactly the product in the phase-one monoid. This family is closed on fixed S, V; it is block triangular rather than diagonal.

Lean witnesses: `RW.toAffine_apply; RW.toAffine_thenOp; RW.toAffine_identity; RW.composeAll_toAffine; Ev.step_eq_summary`

Payoff theorem. Every chronological list of scheduled events has a single affine summary. Sequential execution, arbitrary chunking, every prefix, and any ordered reduction tree give the same state and residual. Empty chunks and prefixes are included. This is exact real arithmetic with no reordering of events.

Lean witnesses: `Ev.combined_scan_compatible; Ev.combined_fold_scan; Ev.combined_closed_summary; Ev.combined_chunked_scan; Ev.combined_prefix_scan; Ev.combined_ordered_tree`

Concrete instance. `E.Write` wraps the actual phase-one `EmenderMemory.TokenDecision`, including its five-operation semantics, while preserving context/world tables. `E.attend` reads the current slot values and the explicit external tables through the tagged position space. Thus a write can change a later STORE read; read values are not merely pre-evaluated constants disguised as a scan.

Lean witnesses: `E.attend_current_values; E.attend_store_source; E.store_index_single_scan; Ev.step_eq_summary`

Schedule-generation boundary. `Ev.combined_routed_scan` additionally takes the existing `TraceMatches` hypothesis: each recorded operator equals the decision computed on the original trajectory. This proves replay equivalence, not that the trace can be generated without traversing that trajectory. The state-dependent support counterexample prevents removing this distinction in general.

Lean witnesses: `Ev.combined_routed_scan; O.state_dependent_support_not_affine`

> **Assumption / unproved refinement.** Match, seed selection, chain DP, correspondence, fork aggregation, fan-out/ranking, value production, and all write gates must supply the schedule. Their state-dependent dependencies remain outside the scan theorem. No theorem here asserts that each such computation is inherently serial, or that chunking reduces its depth. Output tables can be included in V to record separate reads, but their storage and work must then be charged.

7. Named obstructions and the precise closure boundary

State-dependent weights. Two fixed eligible atoms have weights $(1, x)$ and values $(0, x)$. Their candidate-normalized readout is $f(x) = \frac{x^2}{x+1}$. The construction is an actual `AttentionOperator` for $x \geq 0$. At $0, \frac{1}{2}, 1$, all values and weights are nonnegative, but $f(1) + f(0) = \frac{1}{2} \neq \frac{1}{3} = 2f(\frac{1}{2})$. This violates the necessary affine identity and models the problem with recomputing slot-strength normalization from scanned state.

Lean witnesses: `O.strength_read_is_attention; O.state_dependent_weights_witness; O.state_dependent_weights_not_affine`

State-dependent α. A unit singleton read of x, scaled by $\alpha(x) = x$, gives x^2. It is not affine even for $x \in [0, 1]$. In contrast, α held in the supplied schedule is covered by the central linearity theorem. The distinction concerns which coordinates vary, not whether training once produced the coefficient.

Lean witnesses: `O.adaptive_alpha_formula; O.state_dependent_alpha_not_affine; O.fixed_learned_alpha_linear`

State-dependent support and nonlinear values. Choosing between singleton values x and $-x$ according to the sign of x gives $|x|$, which violates the affine identity at $-1, 0, 1$. Even a fixed unit singleton read of a value producer x^2 is non-affine in x. A fixed read does not make its upstream producer affine.

Lean witnesses: `O.routed_support_formula; O.state_dependent_support_not_affine; O.nonlinear_value_source_not_affine`

Diagonal obstruction. On two real coordinates, preserving one coordinate and adding it to the other is a read shear. No diagonal affine operator represents it: at zero, the second coordinate's injection must be zero, while a unit change only in the first coordinate must change that second output. Diagonal maps cannot do that. The closed triangular summary on the enlarged state remains valid.

Lean witnesses: `O.read_not_diagonal; RW.toAffine_thenOp`

Bounded-support obstruction. For every candidate budget B, the sum of $B + 1$ distinct singleton value probes cannot be expressed as a weighted read with at most B atoms on the same value coordinates, for all value tables. Testing each one-hot value table forces every position into the purported smaller support; its cardinality must therefore exceed B.

For every positive budget, two individually budget-admissible nonnegative reads have a sum outside that budget: take B unit atoms and one distinct unit atom. The zero-budget family is closed and is explicitly excluded from this statement.

Lean witnesses: `O.bounded_support_not_closed; O.positive_budget_not_additively_closed; O.zero_budget_iff; O.zero_budget_add_closed`

Omitted-coordinate obstruction. No function of an unchanged store alone can recover arbitrary external values that were omitted from its state. The singleton read can return 0 or 1 for the same store, a contradiction. Carry the value table, or supply a fixed external payload in the schedule.

Lean witnesses: `O.omitted_values_obstruction`

These are precise support and coordinate obstructions. They do not prove absence of all closed families: fixed-dimensional affine and block triangular families are closed. "Compact" here means representation size, not topological compactness. No impossibility theorem about arbitrary nonlinear encodings is asserted.

8. What this does not cover, and validation

Unproved refinements and limitations.

1. **Search and completeness.** No proof of MEM completeness, the registered transformation class, seeds-to-chain alignment, fork correctness, ranking, causal eligibility, document exclusion, or physical index maintenance. R2.3′s prose completeness claim is not established by this algebra.
2. **Exact metadata implementation.** Finite sums and set cardinalities are the specification. No fast index-aggregation algorithm, entropy calculation, provenance deduplication procedure, or truncation selection policy is verified.
3. **Value and schedule production.** Nonlinear neural blocks, MLPs, hidden-state production, attention feedback into new decisions, live/death decisions, and learning are not made parallel by conditioning on their outputs. Matching a supplied trace is a hypothesis, not an algorithm for producing it.
4. **Original coordinates.** Context/world values are carried explicitly in the concrete instance, while slot numeric coordinates come from phase one. Scalar strengths used for weights are frozen into the read schedule. Their recomputation from a changing state is not an affine step in general.
5. **Numerics.** All proofs use exact reals. There is no floating-point reassociation, stability, gradient, zero-denominator implementation, or kernel-parity proof.
6. **Cost.** There is no measured or proved work, span, latency, bandwidth, or memory improvement. A summary map can be large; support can grow with the schedule. Finite-dimensional closure is not an efficient implementation claim. No GPU code or training is included.
7. **Scale bounds and quality.** The norm theorem assumes bounded values and a fixed nonnegative α. Small initialization is not a uniform bound during learning. No theorem establishes useful retrieval, approximation quality under truncation, output-distribution calibration, or a model-quality gain.

Verified proof surface. Four new modules under `ElmanProofs/Architectures/`: `AttentionOperator.lean`, `AttentionMixture.lean`, `AttentionScan.lean`, and `AttentionObstructions.lean`. `PaperCore.lean` imports the new closure. `OPERATOR_ALGEBRA_STATUS.md` indexes the named theorems and retained validation.

Validation procedure (execution evidence, not a mathematical theorem). Run `lake build`, `scripts/check_paper_core.sh`, and `lake env lean scripts/audit_operator_algebra.lean` from `formal/lean`. The audit checks all declarations in all twelve operator modules, inventories source theorems and per-module counts, rejects explicit axioms and transitive dependencies beyond `propext`, `Classical.choice`, and `Quot.sound`, and writes `OPERATOR_ALGEBRA_AUDIT.json`. The machine-readable final census is authoritative.

```
SORRY_COUNT=0
SORRY_REASONS=[]
NEW_AXIOM_COUNT=0
NEW_AXIOMS=[]
```

Inspection note. The baseline phase-one `lake build` passed with 2207 jobs. Final build, audit, trust-check, memo compilation, and source-hash evidence are retained under `build_logs/attention_algebra_*`. The proof source and memo are independently hash-bound in the validation manifest. The external sxgc repository and all training data, receipts, and experiment outputs were untouched.